

UNIVERSIDAD FRANCISCO GAVIDIA

FACULTAD DE INGENIERIA Y SISTEMAS

Desarrollo de Aplicaciones Web - Parcial 4

Documento tecnico breve

API REST Segura, con Auditoria, Cache, Manejo de Concurrencia y Documentacion Avanzada

Estudiante	Kevin Pineda
Carrera	Ingenieria en Desarrollo de Software
Asignatura	Desarrollo de Aplicaciones Web
Docente	Wilfredo Benjamin Magana
Tecnologia utilizada	Node.js + Express
Entrega	Repositorio GitHub, README, coleccion Postman/Insomnia y PDF tecnico

Fecha: 02/06/2026

1. Introduccion

El presente documento resume las decisiones tecnicas aplicadas en el desarrollo de una API REST para la gestion de pacientes de un hospital regional. La solucion fue planteada como una API versionada, segura y documentada, utilizando Node.js con Express, autenticacion JWT, control de acceso basado en roles, validacion de datos, auditoria, cache, manejo de concurrencia por version y documentacion OpenAPI.

El sistema se mantiene con almacenamiento en memoria mediante arreglos, debido a que el alcance del parcial prioriza la arquitectura, seguridad, pruebas y comportamiento de la API sobre la persistencia en base de datos.

2. Tecnologia y dependencias utilizadas

Tecnologia / dependencia	Uso dentro del proyecto
Node.js	Entorno de ejecucion del backend.
Express	Framework principal para rutas, middlewares y controladores.
jsonwebtoken	Generacion y verificacion de tokens JWT con expiracion de 1 hora.
express-validator	Validaciones de entrada para pacientes, consultas, estado y autenticacion.
uuid	Generacion de identificadores unicos para pacientes, consultas y eventos de auditoria.
swagger-ui-express y swagger-jsdoc	Documentacion OpenAPI 3.0 disponible en /api-docs.
cors y dotenv	Configuracion de CORS y variables de entorno.
jest y supertest	Pruebas automatizadas de login, seguridad, auditoria, cache, CSV y concurrencia.

3. Arquitectura en capas

La aplicacion se organizo por capas para separar responsabilidades y facilitar mantenimiento, pruebas y extension futura:

- **Routes:** definen las URL disponibles y conectan cada endpoint con sus middlewares y controladores.
- **Controllers:** reciben la solicitud HTTP, extraen parametros y devuelven respuestas con codigos adecuados.
- **Services:** contienen la logica de negocio, reglas de validacion contextual, auditoria, cache e integridad de datos.
- **Repositories:** administran los arreglos en memoria y exponen funciones de busqueda, filtrado y almacenamiento.
- **Models:** definen la forma esperada de Paciente, Consulta y AuditoriaEvento.
- **Middlewares:** protegen rutas con JWT, validan roles, procesan errores y pueden apoyar el cache.
- **Utils:** centralizan utilidades como AppError, paginacion, cache y generacion de respuestas comunes.

4. Decisiones tecnicas tomadas

4.1 Uso de Express y CommonJS

Se eligio Express por su rapidez de implementacion y porque permite construir APIs REST de forma directa mediante rutas, controladores y middlewares. Se mantuvo CommonJS con require/module.exports para reducir configuracion y avanzar mas rapido durante el parcial.

4.2 Almacenamiento en memoria

Se utilizo almacenamiento en memoria con arreglos para pacientes y auditoria. Esta decision simplifica la entrega academica y permite enfocarse en seguridad, validaciones, versionado, cache y pruebas. En produccion, esta capa podria reemplazarse por una base de datos sin cambiar controladores ni rutas.

4.3 Seguridad con JWT + RBAC

El login genera un JWT firmado que incluye usuario, rol e identificador para auditoria. Cada ruta protegida exige Authorization: Bearer <token>. Luego, un middleware de roles verifica si ADMIN, MEDICO, RECEPCION o AUDITOR tiene permiso para ejecutar la accion.

4.4 Auditoria automatica

Se registro un evento al crear, actualizar, eliminar logicamente o cambiar el estado de un paciente. Cada evento contiene usuario, accion, recursold, detalles y timestamp. Esto permite rastrear acciones criticas del expediente clinico.

4.5 Optimistic locking

El campo version evita que dos usuarios sobrescriban cambios al mismo tiempo. En PUT, el cliente debe enviar la version actual del paciente. Si no coincide con la version guardada, la API responde 409 Conflict. Si coincide, se actualiza el expediente y la version aumenta en 1.

4.6 Cache simple de 30 segundos

El listado paginado de pacientes usa cache en memoria durante 30 segundos. La clave toma en cuenta page, size, sortBy, order, estado, especialidad, edadMin, edadMax y diagnostico. Al crear, actualizar, eliminar o cambiar estado, la cache se invalida para evitar datos desactualizados.

4.7 Exportacion CSV

Se agrego un endpoint exclusivo para ADMIN que exporta pacientes activos en formato CSV. La respuesta usa Content-Type: text/csv y Content-Disposition para que el cliente pueda descargar el archivo.

4.8 Manejo de errores con codigos especificos

Los errores usan un formato uniforme con codigo, timestamp, status, error, mensaje, detalle o detalles y path. Esto facilita la depuracion y permite al frontend identificar errores como token ausente, paciente no encontrado, DUI duplicado o conflicto de version.

4.9 Swagger/OpenAPI

La documentacion se expone en /api-docs e incluye modelos, rutas, parametros, seguridad BearerAuth, respuestas de error, busqueda avanzada, exportacion CSV, auditoria y version del paciente.

5. Funcionalidades principales implementadas

- Login con JWT para usuarios ADMIN, MEDICO, RECEPCION y AUDITOR.
- Control de acceso por roles en endpoints protegidos.

- CRUD parcial de pacientes con eliminacion logica.
- Paginacion, filtrado avanzado y ordenamiento multiple.
- Busqueda por nombre, DUI y diagnostico.
- Registro de eventos de auditoria para acciones criticas.
- Consulta de version actual del paciente.
- Actualizacion protegida con optimistic locking.
- Cache de 30 segundos en listado de pacientes.
- Exportacion de pacientes activos a CSV.
- Documentacion Swagger y pruebas automatizadas con Jest + Supertest.

6. Escenarios minimos demostrables

No.	Escenario	Resultado esperado
1	Login con JWT	Devuelve token valido.
2	Acceso sin token	Devuelve 401 Unauthorized.
3	Crear paciente	Se crea paciente y se registra auditoria CREATE.
4	Modificar con version correcta	Actualiza expediente e incrementa version.
5	Modificar con version incorrecta	Devuelve 409 Conflict.
6	Listar pacientes	La primera peticion indica MISS y la segunda HIT de cache.
7	Exportar CSV	Devuelve archivo CSV de pacientes activos.
8	Consultar auditoria	ADMIN o AUDITOR visualiza eventos registrados.
9	Swagger documentado	La documentacion esta disponible en /api-docs.
10	Prueba automatizada	npm test finaliza correctamente.

7. Respuestas a preguntas de reflexion del Parcial 4

¿Como garantizas que la cache no muestre datos desactualizados en un entorno clinico?

La cache se invalida inmediatamente al crear, actualizar, eliminar logicamente, cambiar estado o agregar informacion relevante al paciente. Ademas, se usa un tiempo de vida corto de 30 segundos y una clave que incluye todos los filtros de consulta. En un entorno clinico real, no se cachearian datos criticos sin politicas estrictas de invalidacion; tambien se podria usar Redis con eventos de invalidacion, versionado de registros y reglas para no cachear expedientes sensibles.

¿Que problemas podria traer el optimistic locking si no se maneja bien en el frontend?

Si el frontend no envia la version correcta o no refresca el expediente antes de actualizar, el usuario recibira conflictos 409 constantemente. Tambien podria perder cambios si el frontend ignora el conflicto y reintenta automaticamente. La forma correcta es mostrar un mensaje claro, recargar la version actual, comparar diferencias y permitir que el medico decida si desea aplicar nuevamente los cambios.

¿Que ventajas/desventajas tiene exportar datos desde la API vs desde la base de datos directamente?

Exportar desde la API permite aplicar seguridad, roles, filtros, auditoria y reglas de negocio antes de entregar datos. Tambien evita dar acceso directo a la base de datos. La desventaja es que la API puede consumir mas memoria y CPU si exporta muchos registros. Exportar desde la base puede ser mas rapido para grandes volúmenes, pero aumenta riesgos si se omiten permisos, filtros o trazabilidad.

¿Como extenderias la auditoria si el sistema creciera a 1000 peticiones por segundo?

La auditoria deberia desacoplarse del flujo principal. En vez de guardar eventos directamente en memoria o en la misma transaccion HTTP, se enviarian eventos a una cola como RabbitMQ, Kafka o SQS. Luego, un consumidor especializado almacenaria eventos en una base optimizada para escritura. Tambien se agregarían índices por usuario, acción, recurso y fecha, rotación de logs y monitoreo.

8. Respuestas a preguntas de reflexion de la guia original

¿Que riesgos de seguridad tiene una API sin autenticacion? Menciona al menos tres.

Una API sin autenticacion permite acceso no autorizado a datos sensibles, modificaciones o eliminaciones maliciosas y suplantacion de acciones sin trazabilidad. Tambien aumenta el riesgo de abuso automatizado, extraccion masiva de informacion y ataques contra endpoints administrativos.

¿Cual es la diferencia entre autenticacion y autorizacion? ¿Como lo aplicaron en este proyecto?

La autenticacion verifica la identidad del usuario, por ejemplo mediante login y JWT. La autorizacion determina que puede hacer ese usuario segun su rol. En el proyecto, el middleware JWT autentica el token y el middleware RBAC autoriza el acceso segun ADMIN, MEDICO, RECEPCION o AUDITOR.

¿Por que es importante paginar los resultados? ¿Que pasaria si devolvieran todos los registros de una base de datos de 1 millon de pacientes?

La paginacion reduce carga del servidor, uso de memoria, ancho de banda y tiempo de respuesta. Si se devolvieran 1 millon de pacientes en una sola respuesta, el servidor podria saturarse, el cliente podria bloquearse y la red tardaria demasiado. Tambien seria un riesgo de privacidad por exponer mas datos de los necesarios.

¿Que ventajas ofrece el versionado de API (/v1/, /v2/) para el ciclo de vida del software?

El versionado permite evolucionar la API sin romper clientes existentes. Por ejemplo, una aplicacion movil podria seguir usando /api/v1 mientras el backend desarrolla /api/v2 con nuevos campos o reglas. Esto facilita mantenimiento, compatibilidad y despliegues graduales.

¿Como garantizarian la integridad de los datos si varios medicos actualizan el mismo expediente al mismo tiempo?

Se aplicaria optimistic locking mediante un campo version. Cada actualizacion debe enviar la version leida por el cliente. Si otro medico ya modifico el expediente, la version cambia y la API rechaza la actualizacion con 409 Conflict. Asi se evita sobrescribir informacion clinica reciente.

¿Que estrategias utilizarian para escalar esta API a un entorno de produccion con alta concurrencia?

Se usaria una base de datos real con índices, pool de conexiones, cache distribuida, balanceador de carga, contenedores Docker, logs centralizados, colas para auditoria, paginacion obligatoria, límites de tasa,

monitoreo y separacion de servicios criticos. Tambien seria importante configurar HTTPS, rotacion de secretos y pruebas de carga.

9. Conclusiones

La solucion propuesta cumple con los requisitos principales del parcial al integrar seguridad JWT, RBAC, auditoria, cache, optimistic locking, exportacion CSV, busqueda avanzada, manejo de errores y documentacion OpenAPI. Aunque el almacenamiento en memoria es suficiente para la practica, la arquitectura en capas permite migrar a una base de datos real y escalar la solucion con cambios controlados.

El mayor aprendizaje tecnico es que una API de produccion no solo debe responder endpoints, sino tambien proteger datos, controlar permisos, registrar acciones criticas, evitar conflictos de concurrencia y entregar informacion documentada y verificable.